The Jessamine symbolic regression system and a reliability approach to benchmarking

W. Garrett Mitchener mitchenerg@charleston.edu

Aiden Camilleri camilleriag@g.cofc.edu
Dept. of Mathematics and Statistics, College of Charleston, Charleston, SC, 29424, USA

Aidan Riordan Riordan.a@northeastern.edu
Northeastern University, Boston, MA, 02115, USA

Latherial Calbert lcalbert6@gatech.edu
College of Engineering, Georgia Institute of Technology, Atlanta, GA, 30332, USA

Abstract

Jessamine is a new open-source, evolutionary, symbolic computation package, based on a flat genome architecture rather than expression trees. For this article, it is configured to perform symbolic regression (SR), discovering prediction functions that are linear combinations of general expressions of controllable complexity. Recombination is straightforward and harmonious with that linear form. Mutation can create useful terms in different individuals, which are recognized as such by selection, and eventually combined in a child genome. To test Jessamine, it was evaluated on data sets chosen from several recent benchmark suites, using a procedure based on SRBench (La Cava et al., 2021). We demonstrate the drawbacks of tuning the hyper-parameters of an SR algorithm on noise-free data, then running it on noisy data, and conversely. Many published benchmarks do so, and we conjecture that this mistuning contributes to reports of disappointing performance. Jessamine performs well on the Strogatz data sets from SRBench, Friedman data sets similar to those in SRBench, and the F corpus from cp3-bench (Thing and Koksbang, 2025). Its performance on the C corpus from cp3-bench is not as strong. Fair comparisons are difficult because of mistuning and conflicting resource constraints, but Jessamine's performance is strong overall. We report results in terms of reliability, a Bayesian estimate of the probability that Jessamine will recover a ground-truth function given limited resources, and encourage the community to adopt similar reporting standards.

Keywords: symbolic regression, evolutionary algorithm, linear genome, benchmark, reliability

1 Introduction

1.1 Interpretable machine learning and symbolic regression

Symbolic regression (SR) is a family of interpretable machine learning (ML) techniques for constructing functions in exact symbolic form as compositions of functions from a given inventory (Rudin et al., 2022; Angelis et al., 2023). In this article, we describe

©201X by the Massachusetts Institute of Technology

and test a new, flexible, evolutionary SR package called Jessamine. It uses a flat genome architecture to represent a collection of functions which are assembled into a prediction function by linear regression.

The problem of finding the best function in symbolic form for a particular data set is ill posed and fraught with statistical and theoretical complications, particularly when the data is noisy. Jessamine's performance is evaluated on a suite of benchmark data sets from the SR literature, with an emphasis on exact recovery of ground-truth expressions from noise-free data. A discussion of these issues, an explanation of the choices of data sets, and detailed results are given in section 3.

Using a procedure and hyper-parameters based on SRBench (La Cava et al., 2021), Jessamine recovers all but one of the functions in the Strogatz data sets from SRBench, and it can recover the remaining one when configured differently. It can recover the Friedman f_1 function, even when the data includes irrelevant columns. It can recover all but two of the F functions from `cp3-bench` (Thing and Koksbang, 2025), and a few of the C functions. Perfectly fair comparisons are difficult, but these results are favorable compared to other algorithms in the literature.

When conducting benchmarks, it is typical to report a single measure of an algorithm's best result on each data set, such as mean-squared error, R^2, or AUC (Aldeia et al., 2025; La Cava et al., 2021; Thing and Koksbang, 2025). Given that evolutionary algorithms are stochastic and resource intensive, we report benchmark results in terms of reliability, that is, what is the probability that an algorithm succeeds given limited computational resources.

A wide range of benchmarks show that no existing algorithm on its own is successful on every type of SR problem, nor is any single configuration likely to be widely successful. We therefore report which types of problems Jessamine works well on, and propose explanations for why it struggles on others. We conclude with suggestions on how to improve benchmark data sets, procedures, and reporting of results, and propose further experiments.

1.2 Background

Linear regression and its generalizations can be understood as forms of SR in which the prediction is a linear combination of a *library* of *derived features*, each of which is a fixed function of baseline features. In Makke and Chawla (2024), this format is called *linear symbolic regression.*

In the dynamical systems community, a popular technique for identifying differential equations from sample trajectories is SINDy, short for sparse identification of nonlinear dynamics (Brunton et al., 2016; Kaheman et al., 2020; de Silva et al., 2020). It finds a function that predicts $x'(t)$ as a linear combination of feature functions $\phi_j(x(t))$, with the ϕ_j's drawn from a fixed library.

The ITEA algorithm searches for linear combinations of terms consisting of a unary function applied to a product of some subset of the inputs, chosen by an evolutionary algorithm (de França and Aldeia, 2021). It can consider more general models than SINDy, but still of a restricted form.

Jessamine goes beyond SINDy by evolving derived features that are not restricted to polynomials. The trade-off is that SINDy is generally fast and does not require sig-

nificant computational resources. Jessamine and other SR algorithms can take several minutes or hours in return for the power to consider a larger class of functions. Like ITEA and Multiple Regression Genetic Programming (MRGP) (Arnaldo et al., 2014), a single Jessamine genome represents many functions of the inputs, and linear regression is used to combine them into a prediction function. It can consider a larger set of expressions than ITEA.

In linear genetic programming (LGP) (Brameier and Banzhaf, 2007), each genome is a sequence of instructions similar to machine language rather than an expression tree (Koza, 1992; dal Piccol Sotto and de Melo, 2016). To avoid overloading the term *linear,* such genome architectures will be called *flat*. LGP and other flat representations have the advantage that recombination is especially straightforward (Randall et al., 2022). Jessamine uses an unordered flat architecture, in which genomes consists of lists of simplified machine language similar to the static-single-assignment form (SSA) generated as an intermediate stage by compilers (Appel, 2004).

1.3 Overview of Jessamine

Training a regression model requires a table $\mathbf{x}$ of points (rows) with numerical baseline features (columns) and a column of target values y. Each Jessamine genome represents a mapping from baseline features to derived features. A selection-mutation process seeks genomes that compute derived features for which some linear combination yields good predictions.

Formally, Jessamine seeks models of the form

$$\hat{y}(\mathbf{x}; b, p) = b_0 + b_1\zeta_1(p, \mathbf{x}) + \cdots + b_K\zeta_K(p, \mathbf{x}) \tag{1}$$

where p and b are arrays of scalars, and ζ is an array of functions that map baseline features to derived features. The algorithm finds an array of functions ζ and numerical arrays p and b that minimize an objective function consisting of a norm of the residual vector $\hat{y} - y$ plus regularization terms. The form (1) biases Jessamine in favor of expressions that are sums of terms at the outermost level of nesting.

Three layers of optimization are performed to find $\hat{y}$. First, given p and ζ, the b vector that minimizes an objective function $R(b, p, \zeta)$ can be found using any form of linear regression with library $\{\zeta_1(p, \mathbf{x}), \ldots \zeta_K(p, \mathbf{x})\}$; ridge regression is used in this article.

In the second layer, with fixed ζ_k's, a general-purpose optimization algorithm finds a p that, together with the corresponding b found by the first layer, yields a local minimum (p, b) for R.

The third layer is that given an inventory of mathematical functions and hyperparameters, a selection-mutation process searches populations of genomes representing sets of feature functions ζ_k. For each genome, it solves for optimal p and b, and uses the resulting value of the objective function to determine the rating $R(b, p, \zeta)$. The selection process generates new genomes from the current generation, favoring those with better ratings. The mutation process introduces small changes to new genomes. Eventually, the population discovers genomes that, along with the corresponding optimal p and b, result in low values for R.

Thus, Jessamine combines the statistical techniques of basis expansion and subset

selection. A population effectively draws derived features from a huge set of possible symbolic forms, but only considers small subsets of them (one genome) at a time.

1.3.1 Genome architecture

Jessamine genomes are in a flat format somewhat different from LGP and ITEA. Leveraging ideas from simulations of artificial regulatory networks (Mitchener, 2014), each individual agent has a state array, some elements of which are constant parameters and inputs. The rest are outputs and scratch space. The agent's genome specifies a mapping from one state array to another. For each output and scratch element, the genome includes an *instruction block* that specifies a function of elements of the state array, the output of which becomes the future value of that element. This mapping is iterated a configurable number of times, called *genome time steps*. The output elements from the final iteration are used as the derived features.

1.3.2 Rationale

The general linear form (1) is practical and matches scientific intuition. Many human-generated symbolic models are sums at the top level, so it makes sense to bias an SR algorithm toward such forms. Evolutionary searches work especially well when fitness includes some kind of partial credit, that is, a fitness improvement for a partial solution, which (1) allows for. If a genome includes a correct function ζ_k, that fact will be discovered by ridge regression, resulting in a good rating for that genome and increased probability of reproduction. Different genomes in a population may have different correct terms. Recombination facilitates the discovery of child genomes that inherit different correct ζ_k's from each parent, thereby making progress toward a fully correct prediction function.

1.3.3 Implementation

In section 2, the operational details of Jessamine are given. The implementation is in Julia, posted as open-source software at `https://github.com/wgm-applied-math/Jessamine.jl`. The action of a genome on a table of data can be computed efficiently, and symbolic expressions for the derived features can be readily constructed using a computer algebra package such as `Symbolics.jl` (Gowda et al., 2022) or SymPy (Meurer et al., 2017). A package is under development that will allow regression and classification algorithms in the `MLJ.jl` toolkit (Blaom and Vollmer, 2021) to be combined with Jessamine, and a Python interface is under development.

2 How Jessamine works

2.1 Genome layer

2.1.1 Specification

Each individual agent in a population has a genome and a state array A that consists of input values $\mathbf{x} = (x_1, \dots, x_N)$, scalar parameters $p = (p_1, \dots, p_L)$, scratch variables $\mathbf{t} = (t_1, \dots, t_M)$, and output variables $\mathbf{z} = (z_1, \dots, z_K)$. The inputs, scratch, and output variables are all of the same type, either scalars or column vectors of compatible sizes

that are processed element-wise. Training points are numbered $j = 1, \ldots, J$ and the row in the training array corresponding to point j is $(x_1(j), \ldots, x_N(j))$ corresponding to the target value $y(j)$. The state array can be interpreted as four sub-arrays $A = (\mathbf{z}, \mathbf{t}, p, \mathbf{x})$ or as a single workspace array formed by concatenating the four sub-arrays in that order.

An agent's genome consists of an array of instruction blocks, one for each scratch and output variable. Each block is a list of instructions. Each instruction specifies a unary function f, a variable-arity aggregation function g, and a list of operand indices. Shorthand notation for an instruction and its mathematical meaning are as follows:

$$\texttt{f@g r}_\texttt{1} \ \ldots \ \texttt{r}_\texttt{H} = f\Big(g\left(A_{r_1}, \ldots, A_{r_H}\right)\Big) \tag{2}$$

Each operand r_h is an index into the state array, so it refers to an z_k, a p_l, a t_m, or an x_n. The aggregation function g must return a result for any list of operands of any length, in any order. The operands may be a mix of scalars and vectors, in which case each scalar is broadcast, meaning it is replaced with a vector of the appropriate size, all elements of which are equal to the original scalar value. The result of an instruction block is the sum of the results of each of its instructions. Overall, a genome specifies an array of functions

$$\big(Z_1(\mathbf{z}, \mathbf{t}, p, \mathbf{x}), \ldots, Z_K(\mathbf{z}, \mathbf{t}, p, \mathbf{x}), T_1(\mathbf{z}, \mathbf{t}, p, \mathbf{x}), \ldots, T_M(\mathbf{z}, \mathbf{t}, p, \mathbf{x})\big)$$

where each Z_k and T_m is defined by the corresponding instruction block.

Evaluation of a genome on given inputs $\mathbf{x}$ and parameters p proceeds as follows. An initial state array $A[0]$ is constructed with $\mathbf{z}$ and $\mathbf{t}$ initialized to zeros, and $\mathbf{x}$ and p inserted in the appropriate slots. Then a specified number S of genome time steps are performed. During genome time step s, all functions Z_k and T_m are evaluated using elements of the current state array $A[s]$ as operand values. The results from those instruction blocks form a new state array, $A[s+1]$, which will be read during the next iteration. That is,

$$\begin{aligned}
z_1[s+1] &= Z_1(\mathbf{z}[s], \mathbf{t}[s], p, \mathbf{x}) \\
&\vdots \\
z_K[s+1] &= Z_K(\mathbf{z}[s], \mathbf{t}[s], p, \mathbf{x}) \\
t_1[s+1] &= T_1(\mathbf{z}[s], \mathbf{t}[s], p, \mathbf{x}) \\
&\vdots \\
t_M[s+1] &= T_M(\mathbf{z}[s], \mathbf{t}[s], p, \mathbf{x}) \\
A[s+1] &= (\mathbf{z}[s+1], \mathbf{t}[s+1], p, \mathbf{x})
\end{aligned}$$

The p and $\mathbf{x}$ arrays are held constant during this iteration. The overall output of the genome is the array $\zeta(p, \mathbf{x}) = (z_1[S], \ldots, z_K[S])$ from the last genome time step. Note that the final result is independent of the positions of instructions within a block and of the order in which they are evaluated.

Some variable-arity functions, like add and multiply, produce the same result independent of the order of their operands. Some take any number of operands but the result depends on their order. Specifically, given operands $u_1, \ldots, u_M$, subtract returns $u_1 - u_2 - \cdots - u_M$, and divide returns $u_1/(u_2 \cdots u_M)$. All of these functions return an appropriate default value (0 or 1) when given no operands.

element	instructions	meaning
$A_1 =$	`mul 4 2`	$Z_1 = p_2 t_1$
$A_2 =$	`exp@mul 3 5 5`	$T_1 = \exp p_1 x_1 x_1$

Table 1: Genome that computes (3).

time step	A_1 z_1	A_2 t_1	A_3 p_1	A_4 p_2	A_5 x_1
0	0	0	$-1/2$	$1/\sqrt{2\pi}$	x
1	0	$\mathrm{e}^{-x^2/2}$	$-1/2$	$1/\sqrt{2\pi}$	x
2	$\frac{\mathrm{e}^{-x^2/2}}{\sqrt{2\pi}}$	$\mathrm{e}^{-x^2/2}$	$-1/2$	$1/\sqrt{2\pi}$	x

Table 2: Steps of computation of the genome in table 1.

Possible unary functions include the identity, sign, absolute value, square root, and the elementary transcendental functions. These return a representation of infinity or not-a-number when given an out-of-domain input or other impossible operation.

The parameter S limits the complexity of discovered functions. Each genome time step adds at most one more layer of function applications $f(g(\dots))$ to the overall computation for each output slot z_k. Thus the depth of the expression tree representation of the ζ_k's is at most S layers of unary-aggregation composites.

2.1.2 Example

To demonstrate how the Jessamine genome layer works, consider the density function of the standard normal distribution,

$$\phi(x) = \frac{1}{\sqrt{2\pi}} \mathrm{e}^{-x^2/2} \tag{3}$$

It may be expressed as a composition of the available operations in many ways, for example:

$$\begin{aligned} Z_1 &= p_2 t_1 \\ T_1 &= \exp p_1 x_1 x_1 \end{aligned} \tag{4}$$

A sufficient state array for this implementation of $\phi(x)$ consists of five elements,

$$A = [z_1, t_1, p_1 = -1/2, p_2 = 1/\sqrt{2\pi}, x_1 = x]$$

The corresponding blocks have one instruction each as shown in table 1. Two time steps are required, as shown in table 2. The final output is read from $A_1[2]$, the z_1 element at time 2.

2.2 Parameter selection layers

Fixing ζ and a value of p, the minimum of $R(b, p, \zeta)$ is found by applying the aforementioned linear regression algorithm to choose b. To find p, the current implemen-

tation uses a general-purpose Nelder-Meade optimization algorithm. This algorithm was chosen because it works with non-smooth objective functions. Jessamine can be configured to use non-smooth operations in the genome, such as step functions, and it can be configured to use a non-smooth rating function. For example, ridge regression could be replaced with LASSO. Thus, the objective function R might not be suitable for gradient-based optimization algorithms. Although these uses of Jessamine are beyond the scope of this article, for simplicity and consistency, Nelder-Meade was used for all initial experiments, including these benchmarks.

2.3 Evolution layer

A population of agents develops good solutions to optimization problems by evolving under a selection-mutation process. In Jessamine, overlapping, discrete generations are produced until a stopping criterion is reached.

2.3.1 Initialization

The initial population consists of randomly generated genomes. The number of instructions in each slot is configurable, as is the random number of operand indices included in an instruction. Operand indices are drawn uniformly from the workspace slots for the inputs x_n's and p_l's. Excluding the t_m and z_k slots at this stage generally improves Jessamine's performance, as doing so reduces the occurrence of ζ_n's that evaluate to 0 or 1, and consequently eliminates many instances of division by zero.

Operations are weighted according to a configurable table. Operations are selected with probability inversely proportional to the corresponding weight. This feature is used to bias the search against using too many square roots and transcendental functions. SR algorithms have a tendency to overuse these functions, as seen in tables 8 to 11 in Thing and Koksbang (2025) and noted in de França and Aldeia (2021).

2.3.2 Rating

Each genome is used to build a complete agent, including a numerical rating used as its evolutionary fitness. Since the applications under consideration are machine-learning tasks where the goal is to minimize an objective function, we adopt the convention that a lower rating is better, which unfortunately clashes with the theoretical biology literature in which it is common to speak of higher fitness being better.

For the benchmark problems in this article, the objective function or rating is computed as

$$R(b, p, \zeta) = \mathrm{MSE} + \lambda_b \|b\| + \lambda_p \|p\| + \lambda_{\mathrm{op}} C \tag{5}$$

where MSE is the mean squared error,

$$\mathrm{MSE} = \frac{1}{J} \sum_j \|y(j) - \hat{y}(x(j); b, p)\|^2$$

The λ_b and λ_p terms penalize excessively large parameter values. In the λ_{op} term, C is the total number of operands in all instructions in the genome. The hyper-parameters λ_b, λ_p, and λ_{op} control the strength of the regularization penalties. The ridge hyper-parameter λ_b must be positive, or the process of solving for b can fail. The non-linearity

hyper-parameter λ_p should be positive so that unused slots in the p vector can be driven to 0. The operand hyper-parameter λ_{op} can be zero, but leaving it positive biases the search in favor of smaller genomes that yield simpler symbolic expressions for $\hat{y}$.

In the jargon of evolutionary computing, fitness is scalarized from the accuracy goal MSE and three regularity goals. A future version of Jessamine could produce a fitness vector $(\text{MSE}, \|b\|, \|p\|, C)$ and seek Pareto-optimal genomes instead.

A complete agent includes a genome, the p and b found by the parameter selection layer, and the value of R.

2.3.3 Selection

Both survival and reproduction are subject to selection. A new generation is formed by keeping some of the strongest agents from the current generation, and replacing the rest with random offspring.

Reproduction is binary. Tournament selection is used to choose each of two parents. With a configurable probability, the very best agent is selected as a parent. Otherwise, two candidates are picked from the current generation, uniformly at random. With a configurable probability, the stronger candidate is selected to be a parent, otherwise, the weaker candidate is selected. The same agent can be selected to serve as both parents.

Two parent genomes are combined using a recombination process similar to crossover of natural chromosomes. For each z_k slot and each t_m slot, the corresponding instruction blocks from the two parents are mixed to form instruction blocks for the child genome. Both blocks are split at a randomly determined position, and the first part of one block is concatenated to the second part of the other.

2.3.4 Mutation

Once a genome is built from two parents, it is subject to random mutations, all with configurable probabilities. Each instruction can be deleted or duplicated; duplicates are inserted in the same block, adjacent to the original. Each instruction can also hop, meaning that it is removed from its block and inserted at the end of another block.

The parts of each instruction can change. Each operand index r_j can be deleted or duplicated within its instruction, or randomly changed to any other valid index into A. The unary and variable-arity functions f and g can be swapped for any combination in the configured operation inventory, selected with probability inversely proportional to their weight.

2.3.5 Simplification

A simplification epoch can be run at the end of the evolutionary search, in which duplication mutations are not allowed. During this epoch, the genome can only shrink, yielding simpler symbolic models. Simplification epochs serve as a regularization tool, as they frequently discover smaller genomes with ratings equal to or less than the best found so far.

3 Benchmarks

3.1 Evaluation of symbolic regression algorithms

3.1.1 Theoretical challenges

Defining intuitive and useful metrics for evaluating SR algorithms is challenging. An obvious benchmark task is to create an array of inputs and apply a known ground-truth function to each sample point to produce a target column, then challenge an algorithm to recover that function from the data. Another is to consider its performance on a standard real-world data set. There are many complications to both approaches.

In evaluating Jessamine, emphasis will be placed on synthetic data sets with known ground truth and no random noise. If noise is present, perfect recovery may be practically impossible; in Thing and Koksbang (2025), no algorithms succeeded on data sets with known ground truth and 10% noise. It is also philosophically problematic: The basis of statistical inference is rejection of a hypothesis on the grounds that the value of a test statistic on a data set is extremely unlikely under that hypothesis. A statistical interpretation of an evolutionary algorithm is that it assigns poor fitness ratings to genomes representing rejectable hypotheses, then selection removes some of them from consideration. However, given noisy synthetic data, an infinitude of incorrect hypotheses are similar enough to the ground truth that the data is insufficient to reject any of them. There is no purely statistical basis for including or excluding very small terms, for example, or choosing between a smooth transcendental function and a polynomial approximation. In experiments that include random noise (section 3.2), the best results come from using an objective function where the penalty for complexity is of a similar magnitude to the MSE for the ground truth. Otherwise, Jessamine tends to over fit. That suggests that the noise should somehow be estimated before running SR, which introduces statistical complications beyond the scope of this article.

SR generally considers expressions including unknown numerical constants, so even if an algorithm discovers the correct general form, some numerical error will be introduced while solving for those constants. As an example, if the ground truth is $F(x) = 2x^3$ and the given data is on $\frac{1}{2} < x < 2$, the prediction $\hat{F}(x) = 1.999x^3 + 10^{-10}x^2$ is not exactly the same function, but it is intuitively "close enough." SR predictions are meant to be interpretable, which introduces a human metric that includes such judgment calls. Specifically, SR algorithms are supposed to be biased in favor of simpler symbolic forms, so it makes sense to post-process the raw results and eliminate *fuzz*, that is, terms of small magnitude that are included because the numerical algorithm is unable to determine that a constant should be exactly 0. However, determining what counts as "small magnitude" requires some human intervention. For instance, the $10^{-10}x^2$ term in the previous example is small compared to the $1.999x^3$ term when $x \approx 1$, but if $x \approx 10^{-10}$, then the two terms are of approximately the same magnitude and both must be kept.

Thus, when performing this kind of benchmark, an SR algorithm might some produce some fuzz that should be ignored, and the threshold of what can be considered fuzz depends on the numerical domain on which the model will be used and the accuracy of some solving process.

A further complication is the possibility of systematic approximations. Like any

other machine-learning algorithm, SR requires a trade-off between accuracy and regularity. Regularity means that a simpler model is preferred to a complex one, unless the complex one is significantly more accurate, but "simpler" is not easily defined in the context of SR. Consider the ground truth function $F(x) = \sin x$, with data drawn from $-0.2 < x < 0.2$. An SR algorithm might produce a degree 5 polynomial as output, which is sensible in that sin is well-approximated by an easily-discovered power series near $x = 0$. The function $\sin x$ is arguably simpler because it is expressed by single function, but the polynomial is also arguably simpler in that it is purely algebraic while sin is transcendental. An algorithm with a bias against transcendental functions might prefer the less-accurate polynomial. A *series trap* occurs when populations are prone to using a series approximation of some kind instead of the correct function.

The dual mistake is also possible. As an example, if $F(x) = 1 - \frac{1}{2}x^2 + \frac{1}{24}x^4$ with data drawn from $-0.2 < x < 0.2$, an SR algorithm might prefer $\hat{F}(x) = \cos x$, which has only one operation, even though it is less accurate. When evaluating benchmark performance, the term *cruft* will be used for needlessly complicated expressions that are numerically accurate. For example, if the ground truth includes a polynomial, but the discovered expression uses a transcendental function to approximate it, that approximation is cruft. A *cruft trap* occurs when populations are prone to using a numerically accurate approximation in terms of overly complicated functions.

Natural data sets present further theoretical complications. As an illustration, the Penn Machine Learning Benchmarks corpus (PMLB) (Olson et al., 2017; Romano et al., 2021) includes data sets based on the Titius-Bode law and Hubble's law.

Dating to the late 1700s, the traditional Titius-Bode law predicts the orbital radius of the n-th planet in the solar system as $0.4 + 0.3 \times 2^n$ astronomical units. Orbital radii are known precisely, and the prediction deviates by only about 5% from the measurements. The Titius-Bode formula aided in the discovery of the asteroid belt and the planet Uranus. However, there is no widely-accepted physical explanation for it, and not all planetary systems follow it (Huang and Bakos, 2014). As appealing as the formula is, it might be a coincidence. Furthermore, the data set is small, consisting of 8 points. An SR algorithm may be able to recover the correct function, but the result is potentially meaningless.

In contrast, Hubble's law models cosmic expansion, consistent with general relativity. The Hubble data set in PMLB consists of only 32 points with high noise. A physicist would realize that it cannot support a model more complex than $y = H_0 x$, but an SR algorithm will likely overfit it.

Any ML algorithm must be configured with some accuracy-regularity trade off, and no single configuration can guarantee success on both noisy and noise-free data sets. In particular, no ML algorithm is likely to give satisfactory results on both the Titius-Bode data and the Hubble data without accuracy-regularity hints specific to each data set. As shown in section 3.2, even for the simple ground-truth function $y = \sin x$, the presence of noise requires an adjustment to Jessamine's λ_{op} hyper-parameter.

So for the purposes of this article, the emphasis will be on idealized data sets that avoid these issues.

3.1.2 Data sets and procedures

The source code for these benchmarks is posted at https://github.com/wgm-applied-math/JessamineBenchmark.jl.

In La Cava et al. (2021), many SR algorithms were run on a corpus of ML problems and each was given 8 hours of CPU time per data set. The scripts to run the benchmarks are posted in a GitHub repository named SRBench, and the authors are currently updating it (Aldeia et al., 2025). Each algorithm was configured with an operation inventory consisting of $\{+, -, \times, \div, x^2, x^3, \sqrt{x}, \ln, \exp, \sin, \cos\}$. Other algorithm-specific hyper-parameters were selected by testing many possibilities on a small collection of noisy real-world data sets from PMLB.

For the benchmarks in this article, Jessamine was configured with that same operation inventory. Other hyper-parameters were chosen by trial and error to do well on the noise-free `ode-strogatz` problems from PMLB (La Cava et al., 2021). This configuration is named *SRB*, short for SRBench, and is spelled out in the supplementary files.

Based loosely on La Cava et al. (2021), the primary procedure for each data set is to run 32 samples of the Jessamine executable, called *micro-trials*, each of which runs for 15 minutes on a single core, for a total of 8 core-hours of computation. Each micro-trial includes as many populations as can be run in that time, which depends on the data set and varies from sample to sample. For data sets where the ground truth is known to be a simple polynomial, each micro-trial is run for only 5 minutes because these problems are easy for Jessamine to solve and there was no need to leave it running. Running many micro-trials facilitates estimation of the probability that Jeessamine correctly recovers a ground-truth function in some amount of time.

PMLB includes data sets based on the Friedman functions, and these are included in SRBench. Rather than use the many Friedman data sets from PMLB, one Friedman function, equation (61) from Friedman (1991), is considered. Data sets with the five inputs alone and with five columns of random distractors test Jessamine's ability to find a signal hidden in high-dimensional data.

In Thing and Koksbang (2025), the `cp3-bench` suite includes two corpora of data sets. The F data sets are abstract, with no specific application. The C data sets are based on cosmological models. Jessamine was tested on all of them using the same procedure as for the Strogatz and Friedman data sets.

For some data sets, a modification to the configuration results in a significant performance increase. Such modified configurations are named *CHT*, short for "cheating," since they incorporate hints and human intuition. These are included in the discussion to illustrate how important configuration details can be and to explain why certain functions are more difficult to recover.

During evolution, the complexity measure of a genome is the total number of operand indices. To facilitate comparison with La Cava et al. (2021), a tree-based complexity is reported. The expression output from Jessamine is converted to a SymPy object, and certain substitutions are applied. Then the expression is *defuzzed* by rounding constants within some tolerance of an integer to that nearest integer. Small constants are therefore replaced with an exact 0. SymPy then does only obvious simplifications such as convert terms of the form $0 \times \cdots$ to 0, delete $0 + \cdots$ and $1 \times \cdots$, and convert

$\sqrt{u^2}$ to $|u|$. The number of nodes in the defuzzed expression is then reported as the solution's complexity.

The proper defuzzing tolerance is, unfortunately, a judgment call, and varies from one data set to another for many reasons. See section 3.1.4 of Thing and Koksbang (2025). In these benchmarks, a tolerance of 10^{-5} is used to calculate the reported complexity. The same tolerance is used for determining whether recovery of the ground-truth function succeeded, unless otherwise stated.

An obvious source of fuzz is that the b_k's in (1) are found by ridge regression, and are generally non-zero. Their magnitude depends on λ_b and the magnitude of the ζ_k terms computed by the genome. Increasing λ_b would decrease the b_k's for unneeded ζ_k terms, but it would also shrink the b_k's corresponding to meaningful terms while changing the balance between accuracy and regularity in the score (5), eventually degrading the quality of discovered expressions. In these benchmarks, $\lambda_b = 10^{-12}$ for all data sets.

For each data set, a *reliability* rating is given, estimating the probability that Jessamine recovers the correct function within 160 minutes for simple polynomials, or 8 hours for other functions. These ratings are based on the following Bayesian model. The number n of 5-minute or 15-minute micro-trials is generally fixed at 32. A macro trial is a success if at least one of its n micro-trials is a success. Assume each micro-trial is an independent Bernoulli random variable with probability of success $r \in (0, 1)$. Then the probability of seeing m successes out of n trials is

$$\mathrm{p}\,(m \mid r) = \binom{n}{m} r^m (1-r)^{n-m}. \tag{6}$$

Assume an uninformative, uniform prior distribution for r,

$$\mathrm{p}\,(r) = \chi_{[0,1]}(r). \tag{7}$$

The posterior distribution of r is $\mathrm{Beta}(m+1, n-m+1)$, which has density

$$\mathrm{p}\,(r \mid m) = \frac{\Gamma(n+2)}{\Gamma(m+1)\Gamma(n+m+1)} r^m (1-r)^{n-m} \tag{8}$$

Let r_c be the solution to $\mathbb{P}\,(r \geq r_c \mid m) = 0.95$. The Bayesian statistical jargon is that $[r_c, 1]$ is a 95% credible interval for r. For a fixed k, let s be the Bernoulli random variable that there is at least one success in k additional micro-trials, that is, a future macro-trial is successful. Then $\mathrm{p}\,(s \mid r) = 1 - (1-r)^k$, which is an increasing function of r. Thus, given m, with 95% credibility,

$$\mathrm{p}\,(s \mid m) \geq 1 - (1 - r_c)^k \tag{9}$$

Fixing $n = 32$ and using Mathematica to solve for r_c for various values of m yields a lower bound for the probability of success of a macro-trial with $k = 32$ at 95% credibility, as shown in table 3.

- If at least 24 of the 32 micro-trials are correct after defuzzing, we say that *with very high reliability* Jessamine can recover the correct function with one 5-minute or 15-minute micro-trial, and it is almost certain to do so if given the full 8 hours.

m	prob succ
1	0.281
2	0.517
3	0.689
4	0.802
5	0.873
6	0.918
7	0.947
8	0.965
9	0.976
10	0.984

Table 3: Lower bound on the probability that a macro-trial of 32 micro-trials succeeds, given m successes out of $n = 32$ initial micro-trials, at 95% credibility.

- If at least 8 of the 32 micro-trials are correct after defuzzing, we say that *with high reliability* Jessamine can recover the correct function given the full 8 hours. This corresponds to a probability of success of at least 96.5% for another macro-trial.
- If at least 4 of the 32 micro-trials are correct after defuzzing, we say that *with moderate reliability* Jessamine can recover the correct function given the full 8 hours. This corresponds to a probability of success of at least 80.2% for another macro-trial.

Otherwise, the reliability is stated on a case-by-case basis. On some data sets, it is clear that recovering the ground-truth function is simply too hard for Jessamine. On others, where it gets close just a couple of times, more micro-trials were run for a total of $n = 192$, which yields a better estimate of reliability. For 15-minute micro-trials, 192 corresponds to 48 hours. See table 4 for the lower bounds on the probability of success in 8 hours given m successful micro-trials out of 192.

3.2 The sin corpus

To demonstrate the importance of the λ_{op} parameter and the challenge of noisy data, consider a data set $y(j) = \sin x(j) + \eta(j)$ where each $x(j)$ is drawn uniformly from $[-\pi, \pi]$ and each $\eta(j)$ is normally distributed with mean 0 and standard deviation 0.1, as show in fig. 1. The mean squared error should be about $0.1^2 = 0.01$, and anything significantly smaller indicates over fitting.

Using the SRB configuration, in which $\lambda_{\text{op}} = 10^{-12}$, the results shown in the left panel of fig. 2. Note that most solutions have high complexity and MSE well below the theoretical value of 0.01, which indicates over fitting. Generally, Jessamine does not recover the ground truth $y = \sin x$. Increasing λ_{op} to 10^{-4}, as shown in the center panel of fig. 2, results in lower complexity, but still shows significant over fitting.

Increasing λ_{op} to 10^{-2}, as show in the right panel of fig. 2, makes a huge difference. Now all but one of the 32 samples return an expression of the form $b_1 \sin x + \cdots$, with $b_1 \approx 1$, which is essentially correct. The MSE is higher, but the complexity is much lower, so over fitting is minimal, yielding more intuitively correct results.

m	prob succ
1	0.057
2	0.125
3	0.199
4	0.272
5	0.341
6	0.406
7	0.465
8	0.520
9	0.569
10	0.613
11	0.653
12	0.689
13	0.721
14	0.749
15	0.775
16	0.798
17	0.818
18	0.836
19	0.853
20	0.867

Table 4: Lower bound on the probability that a macro-trial of 32 micro-trials succeeds, given m successes out of $n = 192$ initial micro-trials, at 95% credibility.

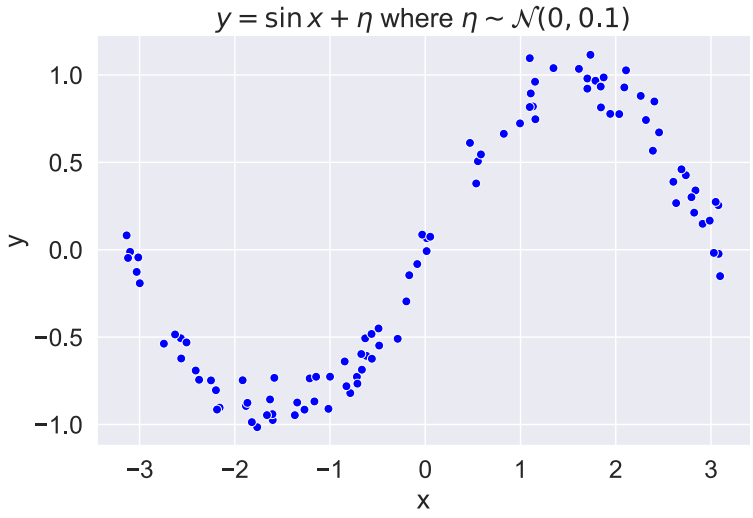


Figure 1: The sin data set.

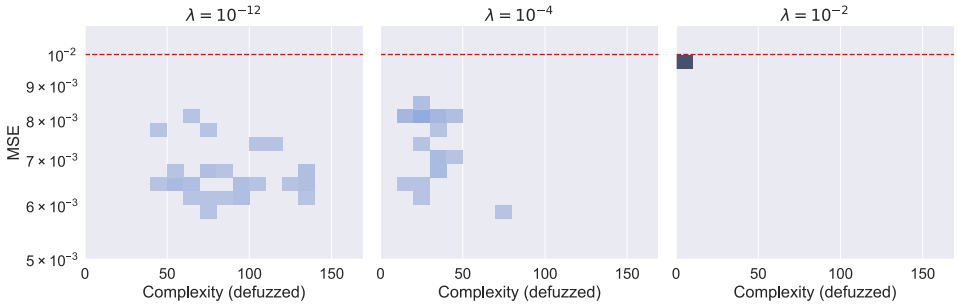


Figure 2: Histogram of results of running Jessamine on the sin data with different values of λ_{op}. Horizontal axis: complexity. Vertical axis: MSE on logarithmic scale. A dashed line is drawn at MSE $= 0.01$.

Thus, when dealing with noisy data, it is important to set λ_{op} so that the complexity penalty in the rating function (5) is approximately the same magnitude as the MSE.

3.3 The Strogatz corpus

The Strogatz corpus, a subset of PMLB, consists of several data sets, each of which consists of trajectories of a two-dimensional nonlinear dynamical system from Strogatz (2014). Each is 400 points, with columns of inputs labeled x and y and a target column of values of one component of the vector field. The task is to infer the vector field functions from the data.

It should be noted that the regression problem here is not inherently dynamic. The points are drawn from trajectories but neither Jessamine nor any of the algorithms in (La Cava et al., 2021) use this fact.

3.3.1 Polynomials

There are two polynomial vector fields. One is a Lotka-Volterra system,

$$\begin{aligned} x' &= 3x - 2xy - x^2 \\ y' &= 2y - xy - y^2 \end{aligned} \tag{10}$$

The other is a Van der Pol oscillator,

$$\begin{aligned} x' &= 10\left(y - \frac{1}{3}(x^3 - x)\right) \\ y' &= -\frac{1}{10}x \end{aligned} \tag{11}$$

After defuzzing at tolerance 10^{-10}, and ignoring tiny imperfections solving for constants, each of the 32 micro-trials of all four data sets correctly finds the ground truth. Jessamine can very reliably recover each of these four polynomials from this data in 5 minutes or less.

3.3.2 Rational functions

Two of the vector fields are rational functions. One is a predator-prey model,

$$\begin{aligned} x' &= x\left(4 - x - \frac{y}{1+x}\right) = 4x - x^2 - y + \frac{y}{1+x} \\ y' &= y\left(\frac{x}{1+x} - 0.075y\right) = -0.075y^2 + y - \frac{y}{1+x} \end{aligned} \tag{12}$$

For the x' function, 5 samples have MSE $< 10^{-26}$ and are clearly symbolically correct after defuzzing at tolerance 10^{-10} and accounting for imperfections in solving for constants.

Another 6 samples have MSE $< 10^{-8}$, and after defuzzing at a tolerance of 10^{-4}, they are very close. To give an example, one sample with MSE $\approx 10^{-8}$ produced, with some rounding,

$$\begin{aligned} \hat{x}' = 4.0007x - 1.0004\,|y| + \frac{1.0006y}{x + 1.0001} \\ - 0.25005\,(-2x - y + 1.0001\,|y| - 0.0002)^2 - 0.0001 \\ + 10^{-5} \cdot (\cdots) \end{aligned} \tag{13}$$

The fuzz elided on the last line includes incorrect terms involving nested transcendental functions, but multiplied by a small constant. Upon telling SymPy to assume that y is positive and defuzzing at tolerance 10^{-4}, the above simplifies to

$$\hat{x}' = -1.0002x^2 - 4x - y + \frac{y}{1+x} \tag{14}$$

which is almost perfectly correct. It seems that the solving process introduces more noticeable numerical discrepancies when solving for constants in a denominator, which increases the MSE and results in the need for a larger defuzzing tolerance to perfectly reconstruct the ground truth.

For the y' function, 3 micro-trials have MSE $< 10^{-26}$ and are clearly symbolically correct after defuzzing at tolerance 10^{-10}. Another 5 samples are correct after defuzzing at tolerance 0.002. So for the predator-prey data sets, Jessamine can recover the correct function with high reliability in 8 hours if the defuzzing tolerance is large.

The other rational-function vector field in the Strogatz corpus is a bacterial respiration model,

$$\begin{aligned} x' &= 20 - x - \frac{xy}{1 + 0.5x^2} = 20 - x - \frac{2xy}{2 + x^2} \\ y' &= 10 - \frac{xy}{1 + 0.5x^2} = 10 - \frac{2xy}{2 + x^2} \end{aligned} \tag{15}$$

These are more difficult for Jessamine to recover exactly. For the x' equation, there is one clearly correct micro-trial when defuzzed at tolerance 0.01. For the y' equation, there is one clearly correct micro-trial when defuzzed at tolerance 10^{-10}, and two more when defuzzed at tolerance 0.01.

To get a better estimate of Jessamine's reliability on these two data sets, a total of 192 micro-trials were run. Among those, for the x' equation, there are three correct micro-trials when defuzzed at tolerance 0.02, and one more that is arguable correct in that it includes terms that approximate

$$\frac{4y}{2x+x^3} - \frac{2y}{x} = -\frac{2xy}{2+x^2}$$

but the constants are far enough away from the correct values that the symbolic form is difficult to recognize. These yield MSE $< 10^{-9}$. Using $m = 4$ in table 4, Jessamine can correctly reconstruct this function in 8 hours with probability 27% or so, with generous defuzzing.

For the y' equation, there are six correct micro-trials when defuzzed at tolerance 0.01, with MSE $< 10^{-11}$. Using $m = 6$ in table 4, Jessamine can correctly reconstruct this function in 8 hours with probability 52% or so, with generous defuzzing.

For many of the original 32 micro-trials for the bacterial respiration data sets, the MSE is quite small, 10^{-8} or less. See fig. 3. These numerically successful micro-trials are generally discovering a series approximation in powers of x^{-1}, which for x' is

$$x' \approx 20 - x - \frac{2y}{x} + \frac{4y}{x^3} + \mathcal{O}\left(x^{-5}\right) \tag{16}$$

It happens that in these data sets, there are no points with small values of x, for which the series yields poor approximations due to the singularity at $x = 0$. Without that evidence that it should reject such a series, the evolutionary process gets stuck in a series trap, a local optimum around the series form.

3.3.3 Trignometric functions

The remaining three pairs of data sets involve trigonometric functions. One pair is a bar magnet model,

$$\begin{aligned} x' &= 0.5\sin(x-y) - \sin x \\ y' &= 0.5\sin(y-x) - \sin y \end{aligned} \tag{17}$$

The second pair is a glider model,

$$\begin{aligned} x' &= -0.05x^2 - \sin y \\ y' &= x - \frac{\cos y}{x} \end{aligned} \tag{18}$$

After defuzzing at tolerance 10^{-10}, and ignoring tiny imperfections solving for constants, each of the 32 micro-trials of all four data sets correctly finds the ground truth. Jessamine can very reliably recover each of these functions in 15 minutes or less.

The remaining pair of data sets is for a shear-flow model,

$$\begin{aligned} x' &= \cot y \cdot \cos x \\ y' &= \left(\cos^2 y + 0.1\sin^2 x\right) \cdot \sin x \\ &= 0.55\sin x + 0.225\left(\sin(x-2y) + \sin(x+2y)\right) \\ &= 0.45\sin x\cos 2y + 0.55\sin x \end{aligned} \tag{19}$$

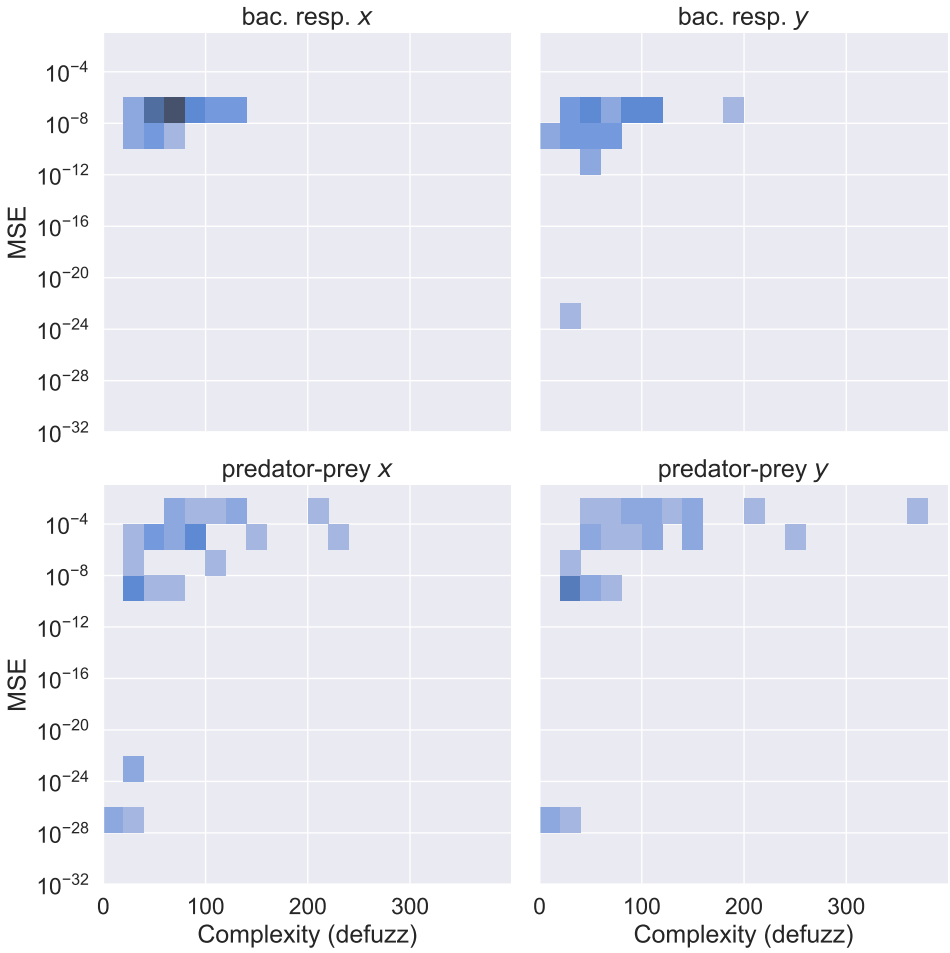


Figure 3: Histogram of results of running 32 micro-trials on the Strogatz rational function data sets. Horizontal axis: complexity. Vertical axis: MSE on logarithmic scale.

The y' function fairly easy to recover. After defuzzing at tolerance 10^{-10}, 17 micro-trials recovered an expression that differs from the original ground truth by a trigonometric identity. So the ground truth is recovered with high reliability in 8 hours.

The x' function is very difficult. Even after a total of 192 micro-trials, the best had MSE $= 7 \times 10^{-5}$ with a symbolic expression that includes $\cos x$ but is otherwise a mess.

The SRB inventory includes sin and cos but no other trigonometric functions, so the task amounts to discovering

$$x' = \frac{\cos y \cdot \cos x}{\sin y}$$

The challenge seems to be the multiplicative nature of the target expression. Jessamine generally discovers the usefulness of $\cos x$, but is unable to discover the rest of the expression. The subexpressions $\sin y$ and $\cos y$ are not obviously useful until exactly the correct form is discovered, which means the evolutionary search is faced with a very rugged fitness landscape and must wait for an unlikely coincidence to reach the best solution.

Note that using a logarithm to transform a multiplicative problem into an additive form might make it easier to solve. However, that approach is not possible here because the factors in the ground-truth function can be negative, and Jessamine is limited to real numbers.

For comparison, a CHT configuration was used in which the operation inventory includes a complete set of the six core trigonometric functions and their inverses. With that configuration, 20 out of 32 micro-trials recover the ground truth, and performance on the y' function is similar to before. So with more functions available than in SRBench, Jessamine can recover both shear-flow functions with high reliability in 8 hours. This example illustrates how important the operation inventory can be.

3.3.4 Summary

A summary of the Strogatz benchmark using the SRB configuration is given in table 5. Given 8 hours, it is likely to recover all but the shear-flow x' function. Adding cot to the operation inventory significantly increases the probability of recovering that one function.

3.4 The Friedman corpus

We consider one of the Friedman functions (Friedman, 1991, eq. 61), commonly used for statistical demonstrations,

$$\begin{aligned} f_1(x_1, x_2, x_3, x_4, x_5) &= 10 \sin(\pi x_1 x_2) + 20 \left(x_3 - \frac{1}{2} \right)^2 + 10 x_4 + 5 x_5 \\ &= 10 \sin(\pi x_1 x_2) + 5 - 20 x_3 + 20 x_3^2 + 10 x_4 + 5 x_5 \end{aligned} \tag{20}$$

Four data sets were constructed using this function; the `fri` data sets from PMLB were not used. Following the `make_friedman1` function from Pedregosa et al. (2011), the data sets were created as follows:

- For f1a, 100 points were drawn at random, with each x_j sampled independently and uniformly from $[0, 1]$.

data set	category	m/n	reliability
Lotka-Volterra	polynomial	32/32	very high (5 min)
Van der Pol	polynomial	32/32	very high (5 min)
predator-prey	rational	8/32	high with generous tolerance
bacterial respiration x'	rational	4/192	low (27%)
bacterial respiration y'	rational	6/192	low (52%)
bar magnet	transcendental	32/32	very high (15 min)
glider	transcendental	32/32	very high (15 min)
shear flow x'	transcendental	0/192	unlikely
shear flow y'	transcendental	17/32	high

Table 5: Summary of the Strogatz benchmark, SRB configuration. For data sets with a percentage in parentheses, this is the estimated probability of at least one correct recovery in 8 hours.

- For f1b, the same process as for f1a was used but 1000 points were drawn.
- For f1c, the same process as for f1a was used to draw 100 points, along with columns $x_6, \ldots, x_{10}$, which are distractors.
- For f1d, the same process as f1c was used but 1000 points were drawn.

No additional noise was added, unlike the `fri` data sets in PMLB.

For these data sets, the best discovered expression have MSE values around 10^{-9}. The discovered expressions must be defuzzed at tolerance 0.02. The need for a numerical approximation to π and the fact that f_1 takes on values up to 30 may explain the larger magnitude of the fuzz compared to, say, the Van der Pol data set.

The results are given in table 6. The additional points present in the f1b and f1d data sets do not improve the results, which is surprising given that the domain of f_1 is five-dimensional. One would expect that a set of 100 points might miss important features that 1000 points are more likely to include. However, f_1 is a sum of terms, only one of which involves two input variables, so the five dimensions are somewhat independent and the additional points do not seem to contribute additional information. Additional time is required to apply a genome to 1000 points compared to 100. That time is largely wasted, leading to fewer genomes explored in the allotted time and weaker performance overall.

To verify this effect, a second batch of 32 micro-trials was run for f1b and f1d with Jessamine configured to evaluate each population on a random subset of 100 of the 1000 points. With this modification, more micro-trials were correct for f1b and f1d as shown in the bottom part of table 6.

As shown by its performance on f1c and f1d, Jessamine is able to find the ground truth even in the presence of distractors. Some of the solutions deemed correct do have terms that reference the extra variables, but with small coefficients. In the overall form (1), the b_k's are found by ridge regression, which is likely to produce a non-zero but small value for the b_k corresponding to an irrelevant variable. Thus, if one of the z_k's is a function of distractors, it will appear in the discovered expression with a non-zero

data set	subset	m/n	reliability
f1a	no	22/32	high
f1b	no	5/32	moderate
f1c	no	6/32	moderate
f1d	no	1/32	unlikely
f1b	yes	19/32	high
f1d	yes	4/32	moderate

Table 6: Summary of the Friedman benchmark, SRB configuration. The subset column indicates whether random subsets were used.

but small coefficient. Defuzzing removes most of these terms.

Some of the discovered expressions with low MSE include cruft. As an example, one micro-trial from f1d resulted in this expression, with constants rounded:

$$\begin{aligned}\hat{y} = {} & 10x_4 + 5x_5 + 10\sin(3.14x_1x_2) \\ & - 40.67\sin(x_3 + 1.07) + 40.68 \\ & + 0.004(\cdots)\end{aligned} \tag{21}$$

Expanding the expression on the second line as a series in x_3 results in

$$6.44 - 21.99x_3 + 17.12x_3^2 + \cdots$$

which is not far from the correct terms

$$5 - 20x_3 + 20x_3^2$$

These cruft expressions seem to be readily discovered because the sample points are drawn from $0 < x_j < 1$, and on that interval, the trigonometric functions are close enough to their degree-2 Taylor polynomials to confuse Jessamine, catching the population in a cruft trap.

3.5 The `cp3-bench` benchmark

The `cp3-bench` benchmark includes two collections of data sets. The F1 to F8 data sets are based on functions listed in Makke and Chawla (2024), which are themselves from other benchmark collections, but the input variables are chosen from different domains. In Thing and Koksbang (2025), the F data sets were used to tune the hyper-parameters of the available SR algorithms for use on the C data sets. The C data sets were created specifically by Thing and Koksbang (2025) based on cosmological phenomena. For some, a ground-truth formula is known, but not for all.

Some of these data sets include tens of thousands of points, in which case Jessamine is sometimes unable to finish more than a generation or two in a 15 minute micro-trial. So for all of these data sets, Jessamine is configured to consider a random subset of 400 points unique to each population. That size matches the Strogatz data sets, which all consist of 400 points. Reported MSE values are computed using the entire data set rather than a subset.

3.5.1 Polynomials F_1 and F_4

There are two polynomial data sets,

$$\begin{aligned} F_1(x) &= x^5 - 2x^3 + x \\ F_4(x,y) &= 2.5x^4 - 1.3x^3 + 0.5y^2 - 1.7y \end{aligned} \tag{22}$$

When defuzzed at tolerance 10^{-6}, all micro-trials for F_1 are correct. When defuzzed at tolerance 5×10^{-4}, all micro-trials for F_4 are correct. So for these two data sets, Jessamine recovers the ground truth with very high reliability within 5 minutes.

3.5.2 The rational function F_7

There is one rational function data set,

$$F_7(x,y,z) = 0.23 + 14.2 \cdot \frac{x+y}{3z} \tag{23}$$

When defuzzed at tolerance 10^{-10}, Jessamine finds the correct function in all 32 micro-trials. So Jessamine recovers the ground truth with very high reliability in 15 minutes.

3.5.3 Power function F_8

There is one data set involving a non-integer power,

$$F_8(x,y,z) = 6.78 + 11\sqrt{7.23xyz} \tag{24}$$

After defuzzing at tolerance 10^{-10}, 29 micro-trials resulted in correct expressions, and the remaining three were correct up to fuzz of up to 10^{-6}. So Jessamine recovers the ground truth with very high reliability in 15 minutes.

3.5.4 Transcendental function F_5

There is one data set with transcendental functions,

$$F_5(x,y) = 1.5\mathrm{e}^x + 5\cos y \tag{25}$$

When defuzzed at tolerance 10^{-10}, Jessamine finds the correct function in all 32 micro-trials. So Jessamine recovers the ground truth with very high reliability in 15 minutes.

3.5.5 Transcendental function F_2

The data set generated by

$$F_2(x) = 0.3x \sin 2\pi x \tag{26}$$

is particularly difficult. The closest Jessamine came to recovering it was by using nearly-resonant cosine terms to approximate $x \sin 2\pi x$. The challenge here seems to be that F_2 is an even function, and neither x nor $\sin 2\pi x$ alone gives a good approximation, as both are odd. Thus, the evolutionary process is unable to notice that x and $\sin 2\pi x$ are potentially useful components.

data set	category	m/n	reliability
F_1	polynomial	32/32	very high (5 min)
F_2	transcendental	0/32	unlikely
F_3	transcendental	0/32	unlikely
F_4	polynomial	32/32	very high (5 min)
F_5	transcendental	32/32	very high (15 min)
F_6	transcendental	0/32	unlikely
F_7	rational	32/32	very high (15 min)
F_8	power	32/32	very high (15 min)

Table 7: Summary of the `cp3-bench` F corpus, SRB configuration.

3.5.6 Transcendental functions F_3 and F_6

The remaining functions

$$F_3(x) = x^3 \cdot \mathrm{e}^{-x} \cdot \cos x \cdot \sin x \cdot \left(\sin^2 x \cdot \cos x - 1\right) \tag{27}$$

and

$$F_6(x) = \frac{\exp\left(-(x-1)^2\right)}{1.2 + (y-2.5)^2} \tag{28}$$

are hopeless. Jessamine did not get close. Neither did any of the algorithms tested in Thing and Koksbang (2025).

3.5.7 Summary of the F corpus

Jessamine did well on F_1, F_4, F_5, F_7, and F_8. It failed on F_2, F_3, and F_6. See table 7. All of the algorithms tested in Thing and Koksbang (2025) failed on F_3 and F_6, and only one was able to recover F_2; none succeeded on more than four.

3.5.8 Polynomial C2a

The C data sets include only one polynomial, C2a, which is a model for measuring redshift drift

$$C_{2a}(z, H) = 0.0716(1+z) - H \tag{29}$$

In all 32 micro-trials, Jessamine correctly recovered this function, so it is found with very high reliability.

3.5.9 Rational C functions

Data set C3g comes from the function

$$C_{3g}(h, v, H) = 6H^2 \frac{v(1-v)(1-h)^2}{(1-v+hv)^2} \tag{30}$$

Jessamine was not able to recover this function. The best micro-trial yielded a complicated expression with MSE $= 2.8 \times 10^{-7}$. In Thing and Koksbang (2025), none of the tested algorithms found the correct function, and the best MSE reported is 4.8×10^{-6}

Data set C3h is a model of the same quantity in terms of other input variables,

$$C_{3h}(a, \ddot{a}, H, \Omega) = -6\left(\frac{\ddot{a}}{a} + \frac{H^2\Omega}{2a^3}\right) \tag{31}$$

Jessamine was not able to recover this function in the first 32 micro-trials. The best micro-trial yielded a complicated expression with MSE $= 1.7 \times 10^{-11}$. In Thing and Koksbang (2025), none of the tested algorithms found the correct function, and the best MSE reported is 5×10^{-6}

When additional micro-trials were run, out of the 192, three recovered the correct function after defuzzing at tolerance 0.01, so the probability of success in 8 hours is about 19.7%.

A CHT configuration was tried in which the operation inventory was limited to addition, subtraction, multiplication, and division. Two micro-trials out of 32 correctly recovered (31) after defuzzing at tolerance 0.001, so this problem can be solved with some hints.

The expression itself is not complicated, so it seems strange that so many SR algorithms are unable to recover it. A possible explanation is that in the data set, $a \in [0.5, 0.9]$, so the singularity at $a = 0$ is not clearly visible. Jessamine is likely getting caught in a series trap around a locally optimal polynomial expansion.

Data set C5a is a dark-matter model,

$$C_{5a}(r) = \frac{1}{r \cdot (1+r)^2} \tag{32}$$

The file includes a column labeled x of all 1's because Thing and Koksbang (2025) considered whether an algorithm could identify it as uninformative and replace it with a literal 1. Since that question is not being considered here, x's in the results are converted to 1 during analysis. When defuzzed at tolerance 2×10^{-6}, 13 micro-trials succeeded, with some algebraic simplifications necessary. Jessamine recovers this function with high reliability in 8 hours.

The data sets C5b and C5c use the same function but add 1% and 10% noise, respectively, and include a constant column of $x = 1$. Jessamine was not able to recover these functions. The MSE of the best micro-trial for C5b is 2.5×10^{-4}, which is greater than the MSE $= 1.0 \times 10^{-4}$ for the correct expression. For C5c, the best MSE $= 0.018$, which is a little more than the MSE $= 0.013$ for the correct expression.

Data set C5d is another dark-matter model,

$$\begin{aligned} C_{5d}(r, R_0) &= \frac{1}{\frac{r}{R_0}\left(1 + \frac{r}{R_0}\right)^2} \\ &= \frac{R_0^3}{r \cdot (r + R_0)^2} \end{aligned} \tag{33}$$

This data set includes a constant column $x = 1$. In one micro-trial, Jessamine recovered this function exactly after algebraic simplification. More micro-trials were run, and with simplification and defuzzing at tolerance 0.002, 5 out of 192 were correct, which means the probability of at least one correct recovery in 8 hours is 34.1%.

Data set C5e is another dark-matter model,

$$C_{5e}\left(r, R_0\right) = \frac{R_0^3}{\left(r + R_0\right) \cdot \left(r^2 + R_0^2\right)} \tag{34}$$

This data set includes a constant column $x = -1$. Jessamine was not able to recover this function. None of the algorithms in Thing and Koksbang (2025) could either. The best MSE $= 1.5 \times 10^{-4}$, which is more than the best reported MSE $= 3.3 \times 10^{-5}$.

Data set C5f consists of a linear combination of C5d and C5e, parameterized by $x = \pm 1$,

$$C_{5f}\left(r, R_0, x\right) = \frac{1}{2} \cdot \left(C_{5d}\left(r, R_0\right) + C_{5e}\left(r, R_0\right)\right) + \frac{x}{2} \cdot \left(C_{5d}\left(r, R_0\right) - C_{5e}\left(r, R_0\right)\right) \tag{35}$$

Jessamine was not able to recover this function. None of the algorithms in Thing and Koksbang (2025) could either.

3.5.10 Power functions

There are several functions related to cosmic expansion and the Hubble phenomenon that have a square root.

$$\begin{aligned} C_{1a}(z) &= 0.0716\sqrt{0.3(1+z)^3 + 0.7} \\ C_{1b}(z) &= C_{1a}(z) + \text{ noise} \\ C_{1c}(z, \Omega) &= 0.0716\sqrt{\Omega \cdot (1+z)^3 + (1-\Omega)} \\ C_{1d}\left(z, \Omega, H_0\right) &= H_0\sqrt{\Omega \cdot (1+z)^3 + (1-\Omega)} \end{aligned} \tag{36}$$

Jessamine was not able to recover these functions, although it finds approximations with MSE-values of 2×10^{-14}, 2×10^{-4}, 1.2×10^{-9}, and 8×10^{-8}, which are not bad. A possible explanation for why these functions are difficult to recover is that on the domains used in these data sets, they are largely featureless. See fig. 4 for a plot of the C_{1a} data. It is a very slight curve, which invites Jessamine and other evolutionary algorithms to start with a linear function, then get caught in a series trap or cruft trap.

Data set C2b also involves a fractional power, this time with a feature value in the exponent.

$$\begin{aligned} C_{2b}(z, \omega) &= 0.0716\left((1+z) - (1+z)^{3(1+\omega)/2}\right) \\ &= 0.0716\left((1+z) - \exp\left(\frac{3(1+\omega)}{2}\ln(1+z)\right)\right) \end{aligned} \tag{37}$$

Jessamine would have to use ln and exp to represent this function, as it has no binary power operation. Jessamine was not able to recover this function, although it found an approximation with MSE $= 1.6 \times 10^{-9}$, which is not bad.

3.5.11 Summary of C1 through C5

See table 8 for a summary of results. Jessamine can recover some of the rational functions with varying reliability, and has no trouble recovering the polynomial. The other data sets are challenging because of noise, the domain, and the symbolic complexity of the ground-truth function.

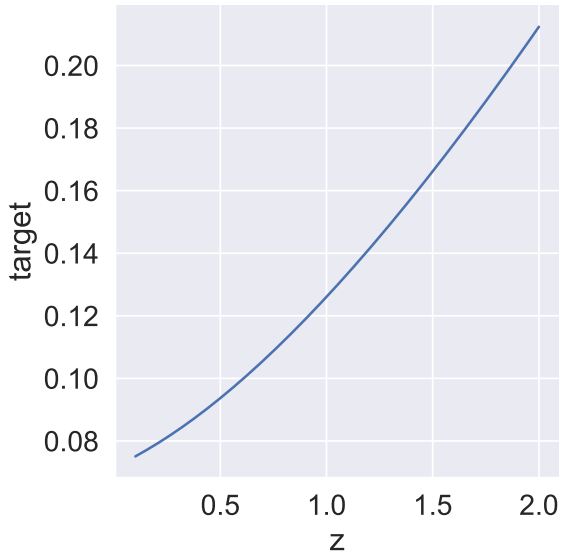


Figure 4: Plot of the C1a data set.

data set	category	noise	m/n	reliability
C1a	power	no	0/32	unlikely
C1b	power	yes	0/32	unlikely
C1c	power	no	0/32	unlikely
C1d	power	no	0/32	unlikely
C2a	polynomial	no	32/32	very high (5 min)
C2b	power/trans	no	0/32	unlikely
C3g	rational	no	0/32	unlikely *MSE
C3h	rational	no	3/192	low (19.7%)
C5a	rational	no	13/32	high
C5b	rational	yes	0/32	unlikely
C5c	rational	yes	0/32	unlikely
C5d	rational	no	5/192	low (34.1%)
C5e	rational	no	0/32	unlikely
C5f	rational	no	0/32	unlikely

Table 8: Summary of the solvable data sets in the C corpus. For data sets marked *MSE, none of the algorithms in Thing and Koksbang (2025) found the correct function, and Jessamine found an incorrect function with MSE less than the best reported in the first 32 micro-trials. For data sets with a percentage in parentheses, this is the estimated probability of at least one correct recovery in 8 hours.

data set	best	target	count
C6a	5.3×10^{-3}	2.2×10^{-3}	0
C6b	8.7×10^{-7}	1.2×10^{-5}	17
C6c	5.1×10^{-1}	1.1×10^{-1}	0

Table 9: Results from the C6 data sets. The "best" column is the lowest MSE from the 32 Jessamine micro-trials. The "target" column is the lowest MSE across all algorithms tested in Thing and Koksbang (2025). The "count" column is the count of micro-trials with a lower MSE than the target.

3.5.12 Hard transcendental functions

The cosmology corpus includes several data sets for complicated expressions involving fractional exponents and transcendental functions. These are C6a, C6b, C6c, and are various forms of a gravitational wave model. None of the algorithms in Thing and Koksbang (2025) recovered these functions. For this collection of data sets, the MSE for the best of the 32 micro-trials is reported in table 9 along with the best MSE reported in Thing and Koksbang (2025).

3.5.13 Black-box data sets

The `cp3-bench` corpus includes two collections of black-box data sets for which no ground-truth function is available. The C3a-f and C4 data sets are for phenomena called cosmic backreaction and redshift drift, respectively. According to Thing and Koksbang (2025); Koksbang (2023a,b), these data sets were synthesized by integrating a system of differential equations, so the goal of SR is to find some relatively simple approximation to the results. The C4 data sets include some noise of unknown magnitude due to various approximations.

For this collection of data sets, the MSE for the best of the 32 micro-trials is reported in table 9 along with the best MSE reported in Thing and Koksbang (2025) and the number of micro-trials for which the MSE was less than or equal to the target.

Numerically, Jessamine produced especially good results for C3b. The best prediction function by MSE is as follows, with the terms ordered roughly from largest to smallest and inexact literal numbers shown at three significant figures:

$$\begin{aligned}\hat{y} = & -0.309\left(1+\mathrm{e}^{-z}\right)^3 - 0.661\mathrm{e}^{-2z+\mathrm{e}^{-z}} + 0.823\mathrm{e}^{-z} \\ & - 0.00862\left(z + \left(1+\mathrm{e}^{-z}\right)^3 + \log(z) + 1.57 + \mathrm{e}^{-z}\right)^2 \\ & - 0.000425\left(\left(1+\mathrm{e}^{-z}\right)^3 + 1.58\mathrm{e}^{-2z+\mathrm{e}^{-z}} + 1\right)^3 \\ & + 0.105\log(z) + 0.0474z + 0.53 \\ & - 0.00182\mathrm{e}^{-2z+1.58\mathrm{e}^{-2z+\mathrm{e}^{-z}}}\end{aligned} \tag{38}$$

All terms but the last line must be included to produce a plot in which the prediction is visually correct. Unfortunately, this expression does not yield much insight. The target function $C_{3b}(z)$, shown in fig. 5, is nearly linear in $(1+\mathrm{e}^{-z})^3$, and the rest of

data set	best	target	count
C3a	1.5×10^{-13}	4.2×10^{-13}	2
C3b	3.9×10^{-14}	3.6×10^{-11}	32
C3c	3.4×10^{-7}	1.4×10^{-7}	0
C3d	1.7×10^{-7}	1.3×10^{-6}	24
C3e	3.0×10^{-7}	4.6×10^{-7}	4
C3f	1.5×10^{-7}	3.8×10^{-6}	32
C4a	2.6×10^{-2}	4.0×10^{-2}	32
C4b	3.7×10^{-2}	3.5×10^{-2}	0
C4c	3.6×10^{-2}	3.6×10^{-2}	0
C4d	1.0×10^{-3}	1.7×10^{-3}	20
C4e	1.2×10^{-4}	4.8×10^{-5}	0

Table 10: Results from the C3 and C4 black-box data sets. The "best" column is the lowest MSE from the 32 Jessamine micro-trials. The "target" column is the lowest MSE across all algorithms tested in Thing and Koksbang (2025). The "count" column is the count of micro-trials with a lower MSE than the target.

the expression for $\hat{y}$ appears to be corrections to this approximation. However, for $z \in [0.2, 1]$ as in this data set, many transcendental functions are well-approximated by power series in z, so, for example, $C_{3b}(z)$ is also nearly linear in $(2 - z + z^2/2)^3$, Thus, it is unclear whether the exponential component is truly meaningful or just cruft approximating something else.

The other numerically accurate expressions for C_{3b} given in Thing and Koksbang (2025) do not have much in common with (38) or with each other, symbolically. Several of them suggest a pole or logarithmic singularity for some z less than 0.2.

4 Discussion

4.1 Jessamine-specific observations

The benchmarking experiments reveal several strengths and weaknesses of the Jessamine system.

Its bias toward linear combinations of functions facilitated recovery of the ground-truth functions in the Strogatz corpus, Friedman corpus, and `cp3-cosmo` F corpus that had that form. Its performance on those corpora was strong compared to other algorithms reported in La Cava et al. (2021) and Thing and Koksbang (2025). A negative consequence of that bias is that it has difficulty recovering some functions with multiplicative structure, such as (26), even though it succeeds on others, such as (32). A possible improvement would be to allow instruction blocks to compute the product of the results of their instructions rather than the sum, or to choose aggregation functions from an inventory, but that would require additional configuration and testing.

Jessamine genomes represent functions with a fixed number $K + 1$ of top-level terms as in (1). Since the coefficients b_k are found by ridge regression in these experiments, terms that are not useful still appear in the final output, just with a small value

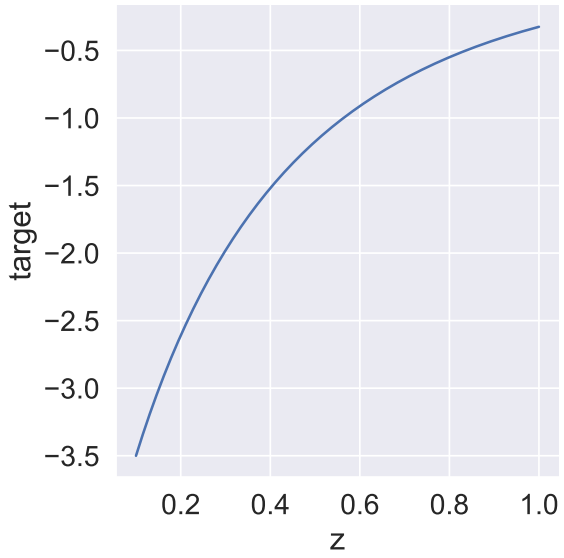


Figure 5: Plot of the C3b data set.

of b_k, resulting in fuzz. A potentially useful modification would be to use a different linear-regression algorithm, such as LASSO, elastic net, or subset selection (Hastie et al., 2009), or to post-process successful genomes found using ridge regression. These eliminate terms from a linear model entirely, however, other hyper-parameters are required to specify how strong this effect should be. It is not immediately clear that choosing these additional hyper-parameters is any better than choosing the defuzzing threshold.

The genome can potentially play tricks that affect the regularity terms in the rating function (5). These terms penalize large values of b_k and p_l, but a genome could use division to bypass this penalty. For example, it could introduce a large coefficient on x_1 by including a term $b_1 x_1 / p_1$ and finding a small p_1. This phenomenon was not observed in these experiments. Other biases toward simple expressions may be discouraging Jessamine from using such tricks. However, evolutionary processes are notorious for taking advantage of quirks in the fitness landscape, so the overall expression encoded by a genome and its parameters should be examined carefully for terms that seem to be of unusual magnitude.

4.2 Configuration

The configuration of an algorithm is extremely important. The sin experiment in section 3.2 shows that for data sets with noise, Jessamine must be configured with greater bias against expression complexity, otherwise it over fits. Presumably the same is true of other SR algorithms. In La Cava et al. (2021), the authors used black-box, real-world data sets for hyper-parameter tuning, which likely yields configurations that are not ideal for use on noise-free data sets. This choice might be why many of the tested algorithms performed poorly on the Strogatz data sets. In Thing and Koksbang (2025), the authors used the F corpus for hyper-parameter tuning, but then found that some

algorithms performed well on the F corpus but poorly on the C corpus. For Jessamine, the SRB configuration used in these experiments performs well except on the C corpus, but these experiments indicate that no single configuration is universally ideal. For any SR algorithm, it is unlikely that a single configuration will give high-quality results on both noisy and noiseless data sets.

The shear-flow experiment from section 3.3.3 shows how important the operation inventory can be. For Jessamine, recovering the x' formula (19) was relatively easy when cot was available, and extremely difficult when it was not. Other algorithms might show the same improvement if allowed to use cot.

Based on these experiments, it seems that an algorithm must perform some kind of adaptive hyper-parameter tuning to produce potentially useful results on both noise-free and noisy data sets. Such a meta-algorithm should run many instances of a fixed algorithm using a range of hyper-parameters, including a range of accuracy-regularity trade-offs, progressively larger operation inventories, and greater resources. The result will be a set of expressions that vary in complexity and accuracy, and some external process must choose among them. It is not clear how limited computing resources should be allocated under such a scheme, such as the 8 hours on a single core used here. Based on these experiments, about 5 minutes should be spent seeking a simple polynomial, and the rest spent on rational and transcendental functions. Further experiments are required.

4.3 Benchmarks, reporting, and comparisons

Overall, Jessamine's performance on ground-truth recovery tasks is strong compared to other SR algorithms. A more exact comparison is difficult. The SRBench results are perhaps the fairest comparison, because all algorithms were run with comparable resource limitations . The results on the Strogatz data sets are reported as aggregations (La Cava et al., 2021, fig. 8), and hyper-parameter tuning was on noisy data, so it is difficult to say which algorithms are generally better and why, or which data sets are generally harder.

For the reported `cp3-bench` results, fair comparisons are more difficult, because the tested algorithms ran for a wide range of times. (The stated purpose of the `cp3-bench` project was to facilitate use of a variety of SR packages and to illustrate some of their strengths and weaknesses, and not primarily to perform completely fair benchmarks.) In some cases, no extra time was needed. For example, the uDSR algorithm (Landajuela et al., 2022) solved C1a perfectly in 13 seconds and did not need additional resources, but other algorithms ran for hours and did not succeed. In other data sets, it is unclear whether a successful algorithm could have stopped early but was configured not to, or whether an algorithm that stopped unsuccessfully after less than 8 hours could have been run again with a different random seed and perhaps succeeded within the remaining time.

Let us consider the `cp3-bench` results in context. Table 4 of Thing and Koksbang (2025) shows that the three best algorithms by number of ground-truth functions correctly recovered were PySR (Cranmer et al., 2020), uDSR (Landajuela et al., 2022), and QLattice (Wilstrup and Kasak, 2021; Christensen et al., 2022). PySR recovered the correct function from 6 of these data sets, and uDSR and QLattice succeeded on 4. All found C2a. PySR found C5a in a minute, and C5b and C5c in under an hour, which

are the same function with different amounts of noise. It found C5d and C5e in under 2 hours. uDSR found C1a, C1c, and C1d in minutes. QLattice found C2b, C5a, and C5b in under 2 hours. PySR succeeded on F1, F5, and F7. QLattice and uDSR succeeded on F1, F4, and F7. Time spent suggests that the reported successes of PySR, uDSR, and QLattice are not simply due to luck, but a reliability estimate would be a stronger measure. Several of these successes are on data sets with definite similarities, which suggests that each algorithm has distinct strengths and weaknesses. PySR seems to do well with rational functions and noisy data. uDSR and QLattice seem to do well on functions with particular algebraic structure.

Jessamine as configured clearly succeeded on C2a and C5a, and succeeds on C3h and C5d given additional time, a weaker performance than the three best on the C corpus. Interestingly, C3h is a rational function, which is not Jessamine's specialty as it is not a linear combination of general functions. No other algorithm in Thing and Koksbang (2025) succeeded on C3h. However, Jessamine succeeded on five of the F problems, which is a strong performance. An overall comparison is difficult.

In the context of Thing and Koksbang (2025); La Cava et al. (2021); Aldeia et al. (2025), these experiments with Jessamine suggest the following recommendations to improve the usability of benchmark results, particularly for SR algorithms that make use of randomness.

- Algorithms should be tuned on data and with noise comparable to data they will be used on. Mismatching hyper-parameter tuning with test data yields disappointing results from all algorithms.
- It makes sense to estimate reliability, that is, the probability that an algorithm will succeed given a fixed amount of computing resources. The Bayesian formulation from section 3.1.2 is one way to do this.
- Alternatively, one could estimate the amount of computing resources needed to recover a ground truth with some confidence. For example, how long must an algorithm run to have a 90% probability of recovering the predator-prey model?
- Ground-truth data sets should include functions with a variety of algebraic structures and complexity, to reveal the strengths and weaknesses of each algorithm.

4.3.1 Obstacles to ground-truth recovery

The difficulty of recovering a ground-truth function increases with the complexity of its algebraic expression, as with (27). As noted in La Cava et al. (2021), some of the tested algorithms consider a limited set of expression structures and cannot represent some of the ground-truth functions in SRBench and `cp3-bench`, such as ITEA, for example (de França and Aldeia, 2021). The choice of available operations is also significant, as with (19).

In addition, properties of the data set itself are important. An evolutionary algorithm can become trapped around a local optimum in the fitness landscape. Data that only includes inputs not far from 0 can lead an algorithm toward series traps and cruft traps.

With that in mind, it makes sense to conduct further experiments along the following lines.

- When the ground truth has a pole or other singularity, try some data sets that avoid the singularity and others that come close to it. The first kind of experiment measures how successful an algorithm is at extrapolating to a singularity. The second kind measures how much trouble is caused by including target values that vary over several orders of magnitude.

- When the ground truth has asymptotic behavior for large or small input, try some data sets that illustrate it and some that obscure it. For example, $1/(1+x^2)$ converges to 0 as $x \to \pm\infty$. Is it harder to recover this function when x is drawn from $[0.5, 1]$ compared to $[-2, 2]$?

- For oscillatory functions, try some data sets that include only part of an oscillation and others that include one or more complete oscillations. These measure how successful an algorithm is at extrapolating oscillations. Some preliminary experiments with the sin function as in section 3.2 showed that Jessamine can recover $\sin x$ from only a small part of a period, but recovery is facilitated because a term of the correct form $z_k = \sin x$ is likely to be present in many random initial populations and discovered immediately. A more algebraically complicated function may yield more interesting results.

- More generally, it is worth exploring how algorithms fall into series traps and cruft traps, as in (16) and (21). Data sets should be built by drawing inputs from various intervals on which functions in the operation inventory are well or poorly approximated by an easily-discovered low degree polynomial. Then one can measure how likely an algorithm is to choose a polynomial or transcendental function.

4.3.2 The partial credit problem

Jessamine recovered the Friedman function f_1 (20) with high reliability (data set f1a). However, the algebraically simple function F_2 (26) was never successfully recovered. The difference seems to be that f_1 is a sum of terms, and each term can be constructed from a single instruction or sum of instructions, exactly the kind of function Jessamine is biased to favor. Each term alone earns partial credit. The terms can arise in separate agents, in which case selection notices that each yields some improvement to the rating. Eventually these agents or their descendants will be selected as parents and produce a child that contains all the terms by recombination.

In contrast, F_2 is a product, and neither factor alone is a good approximation. Thus, when an evolutionary algorithm discovers a genome with just one of the factors, or that has both factors but connected by an operation other than $\times$, a rating based on prediction accuracy gives no evidence that the genome is on the right track. There is no partial credit for this function, and based on table 7 of Thing and Koksbang (2025), most other algorithms are likewise stumped.

This suggests that an evolutionary search may be more successful at recovering functions like F_2 if the fitness function includes measures other than numerical prediction accuracy. Possibilities include locations of zero crossings, oscillatory frequency, envelope, and asymptotic behavior. Further research remains to be done.

References

Aldeia, G. S. I., Zhang, H., Bomarito, G., Cranmer, M., Fonseca, A., Burlacu, B., La Cava, W. G., and de França, F. O. (2025). Call for Action: Towards the next Generation of Symbolic Regression Benchmark. In *Proceedings of the Genetic and Evolutionary Computation Conference Companion*, pages 2529–2538.

Angelis, D., Sofos, F., and Karakasidis, T. E. (2023). Artificial Intelligence in Physical Sciences: Symbolic Regression Trends and Perspectives. *Archives of Computational Methods in Engineering*, 30(6):3845–3865.

Appel, A. W. (2004). *Modern Compiler Implementation in ML*. Cambridge Univ. Press, Cambridge.

Arnaldo, I., Krawiec, K., and O'Reilly, U.-M. (2014). Multiple Regression Genetic Programming. In *Proceedings of the 2014 Annual Conference on Genetic and Evolutionary Computation*, GECCO '14, pages 879–886, New York, NY, USA. Association for Computing Machinery.

Blaom, A. D. and Vollmer, S. J. (2021). Flexible Model Composition in Machine Learning and Its Implementation in MLJ. Preprint arXiv:2012.15505.

Brameier, M. F. and Banzhaf, W. (2007). *Linear Genetic Programming*. Genetic and Evolutionary Computation Series. Springer, New York.

Brunton, S. L., Proctor, J. L., and Kutz, J. N. (2016). Discovering Governing Equations from Data by Sparse Identification of Nonlinear Dynamical Systems. *Proceedings of the National Academy of Sciences*, 113(15):3932–3937.

Christensen, N. J., Demharter, S., Machado, M., Pedersen, L., Salvatore, M., Stentoft-Hansen, V., and Iglesias, M. T. (2022). Identifying Interactions in Omics Data for Clinical Biomarker Discovery Using Symbolic Regression. *Bioinformatics*, 38(15):3749–3758.

Cranmer, M., Sanchez-Gonzalez, A., Battaglia, P., Xu, R., Cranmer, K., Spergel, D., and Ho, S. (2020). Discovering symbolic models from deep learning with inductive biases. Preprint arXiv:2006.11287.

dal Piccol Sotto, L. F. and de Melo, V. V. (2016). Studying Bloat Control and Maintenance of Effective Code in Linear Genetic Programming for Symbolic Regression. *Neurocomputing*, 180:79–93.

de França, F. O. and Aldeia, G. S. I. (2021). Interaction–Transformation Evolutionary Algorithm for Symbolic Regression. *Evolutionary Computation*, 29(3):367–390.

de Silva, B. M., Champion, K., Quade, M., Loiseau, J.-C., Kutz, J. N., and Brunton, S. L. (2020). PySINDy: A Python Package for the Sparse Identification of Nonlinear Dynamical Systems from Data. *Journal of Open Source Software*, 5(49):2104.

Friedman, J. H. (1991). Multivariate Adaptive Regression Splines. *The Annals of Statistics*, 19(1):1–67.

Gowda, S., Ma, Y., Cheli, A., Gwóźzdź, M., Shah, V. B., Edelman, A., and Rackauckas, C. (2022). High-Performance Symbolic-Numerics via Multiple Dispatch. *ACM Commun. Comput. Algebra*, 55(3):92–96.

Hastie, T., Tibshirani, R., and Friedman, J. H. (2009). *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. Springer Series in Statistics. Springer, New York, NY, 2nd edition.

Huang, C. X. and Bakos, G. Á. (2014). Testing the Titius–Bode Law Predictions for Kepler Multiplanet Systems. *Monthly Notices of the Royal Astronomical Society*, 442(1):674–681.

Kaheman, K., Kutz, J. N., and Brunton, S. L. (2020). SINDy-PI: A Robust Algorithm for Parallel Implicit Sparse Identification of Nonlinear Dynamics. *Proceedings of the Royal Society A: Mathematical, Physical and Engineering Sciences*, 476(2242):20200279.

Koksbang, S. M. (2023a). Cosmic Backreaction and the Mean Redshift Drift from Symbolic Regression. *Physical Review D*, 107(10):103522.

Koksbang, S. M. (2023b). Machine Learning Cosmic Backreaction and Its Effects on Observations. *Physical Review Letters*, 130(20):201003.

Koza, J. R. (1992). *Genetic Programming: On the Programming of Computers by Means of Natural Selection*. MIT Press, Cambridge, MA.

La Cava, W., Orzechowski, P., Burlacu, B., de Franca, F. O., Virgolin, M., Jin, Y., Kommenda, M., and Moore, J. H. (2021). Contemporary Symbolic Regression Methods and Their Relative Performance. In *Thirty-Fifth Conference on Neural Information Processing Systems Datasets and Benchmarks Track (Round 1)*.

Landajuela, M., Lee, C. S., Yang, J., Glatt, R., Santiago, C. P., Aravena, I., Mundhenk, T., Mulcahy, G., and Petersen, B. K. (2022). A Unified Framework for Deep Symbolic Regression. In *Advances in Neural Information Processing Systems*, volume 35, pages 33985–33998. Curran Associates, Inc.

Makke, N. and Chawla, S. (2024). Interpretable Scientific Discovery with Symbolic Regression: A Review. *Artificial Intelligence Review*, 57(1):2.

Meurer, A., Smith, C. P., Paprocki, M., Čertík, O., Kirpichev, S. B., Rocklin, M., Kumar, Am., Ivanov, S., Moore, J. K., Singh, S., Rathnayake, T., Vig, S., Granger, B. E., Muller, R. P., Bonazzi, F., Gupta, H., Vats, S., Johansson, F., Pedregosa, F., Curry, M. J., Terrel, A. R., Roučka, Š., Saboo, A., Fernando, I., Kulal, S., Cimrman, R., and Scopatz, A. (2017). SymPy: Symbolic Computing in Python. *PeerJ Computer Science*, 3:e103.

Mitchener, W. G. (2014). Evolution of Communication Protocols Using an Artificial Regulatory Network. *Artificial Life*, 20(4):491–530.

Olson, R. S., La Cava, W., Orzechowski, P., Urbanowicz, R. J., and Moore, J. H. (2017). PMLB: A Large Benchmark Suite for Machine Learning Evaluation and Comparison. *BioData Mining*, 10(1):36.

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., and Duchesnay, É. (2011). Scikit-Learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12:2825–2830.

Randall, D. L., Townsend, T. S., Hochhalter, J. D., and Bomarito, G. F. (2022). Bingo: A Customizable Framework for Symbolic Regression with Genetic Programming. In *Proceedings of the Genetic and Evolutionary Computation Conference Companion*, pages 2282–2288, New York, NY, USA. Association for Computing Machinery.

Romano, J. D., Le, T. T., La Cava, W., Gregg, J. T., Goldberg, D. J., Ray, N. L., Chakraborty, P., Himmelstein, D., Fu, W., and Moore, J. H. (2021). PMLB v1.0: An Open Source Dataset Collection for Benchmarking Machine Learning Methods. Preprint arXiv:2012.00058.

Rudin, C., Chen, C., Chen, Z., Huang, H., Semenova, L., and Zhong, C. (2022). Interpretable Machine Learning: Fundamental Principles and 10 Grand Challenges. *Statistics Surveys*, 16:1–85.

Strogatz, S. H. (2014). *Nonlinear Dynamics and Chaos: With Applications to Physics, Biology, Chemistry, and Engineering*. Westview Press, Boulder, CO, 2nd edition.

Thing, M. E. and Koksbang, S. M. (2025). Cp3-Bench: A Tool for Benchmarking Symbolic Regression Algorithms Tested with Cosmology. *Journal of Cosmology and Astroparticle Physics*, 2025(01):040.

Wilstrup, C. and Kasak, J. (2021). Symbolic regression outperforms other models for small data sets. Preprint arXiv:2103.15147.